The Cost of the Quantum Transition

ECE 4301 Final Project Presentation

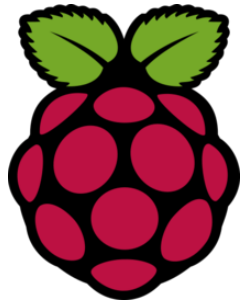
Group D – Joshua Deckman and Jack Pearson

The Goal

- Report where the post-quantum transition will be costly vs cheap
- Compare Diffie-Hellman (pre-quantum) and Kyber (post-quantum)
  - Sensitivity to adverse network conditions
- Anticipation: Kyber much more sensitive to bandwidth, similar sensitivity to delay
- Simulate on Raspberry Pi 5

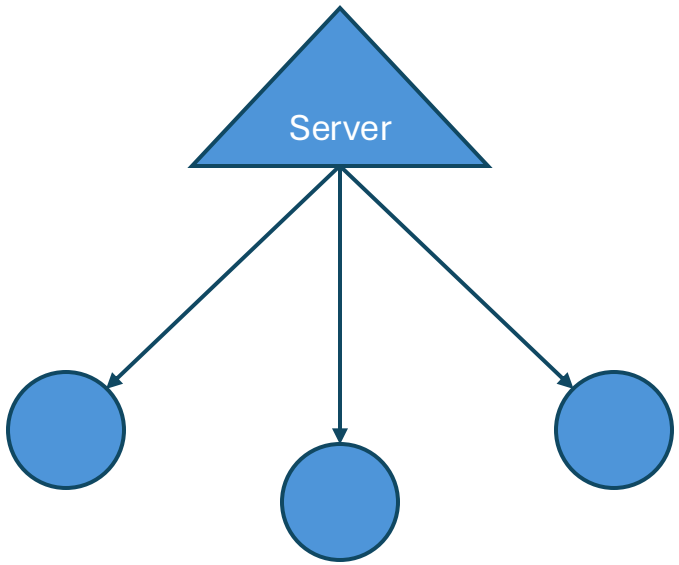
Vs.

ECC Diffie-Hellman



One Executable, Two Modes

- Client/Server Topology
- The client times key exchanges




```rust
fn client(addr: &str, config: Config) {
    let mut stream: TcpStream = TcpStream::connect(addr).unwrap();
    ciborium::into_writer(value: &config, writer: &mut stream).unwrap();
    stream.flush().unwrap();

    match config.mode {
        Mode::BufferExchange => {…
        Mode::DiffieHellman => {…
        Mode::Kyber => {
            let start: Instant = Instant::now();

            // Receive server's public key
            let mut public_key_bytes: [u8; 1184] = [0u8; KYBER_PUBLICKEYBYTES];
            stream.read_exact(buf: &mut public_key_bytes).unwrap();
            debug!("Client received server public key ({} bytes)", public_key_bytes.len());

            // Generate ciphertext and shared secret
            let mut rng: OsRng = OsRng;
            let (ciphertext: [u8; 1088], client_shared_secret: [u8; 32]) = encapsulate(pk: &public_key_bytes, &mut rng).unwrap();

            debug!("Client shared secret: {:?}", client_shared_secret);

            // Send ciphertext to server
            stream.write_all(buf: &ciphertext).unwrap();
            stream.flush().unwrap();

            let elapsed: f64 = start.elapsed().as_secs_f64() * 1000.0;
            println!("Kyber exchange time: {:.3} ms", elapsed);

            // Graceful shutdown: wait for server acknowledgment before exiting
            let mut ack: [u8; 1] = [0u8; 1];
            stream.read_exact(buf: &mut ack).unwrap();
            debug!("Received graceful shutdown ACK from server");
        }
    }
} fn client

fn server(addr: &str) {
    let listener: TcpListener = TcpListener::bind(addr).unwrap();
    info!("Server listening on {}", addr);

    loop {
        let (mut stream: TcpStream, peer_addr: SocketAddr) = listener.accept().unwrap();
        debug!("Connection from {}", peer_addr);

        let config: Config = ciborium::from_reader(&mut stream).unwrap();
        debug!("Received config: mode = {:?}, buffer_size = {}", config.mode, config.buffer_size);

        match config.mode {
            Mode::BufferExchange => {…
            Mode::DiffieHellman => {…
            Mode::Kyber => {…
        }
    }
} fn server
```

Network Namespace

- Wraps client executable
- Network parameters (such as bitrate) can be regulated
- For testing, parameters were swept

```
########################################
# Traffic control: netem (delay + rate)
########################################

# Root namespace side
tc qdisc add dev "$VETH_ROOT" root handle 1: netem \
  limit "$NETEM_LIMIT_PKTS" delay "$DELAY" rate "$RATE"

# Namespace side
ip netns exec "$NS_NAME" tc qdisc add dev "$VETH_NS" root handle 1: netem \
  limit "$NETEM_LIMIT_PKTS" delay "$DELAY" rate "$RATE"

# Finally, run the client binary inside the namespace
ip netns exec "$NS_NAME" "$BINARY_PATH" "$@"
```

Data Acquisition

- Python script to automate acquisition
- Sweeps through specified network parameters
- Writes results to CSV file

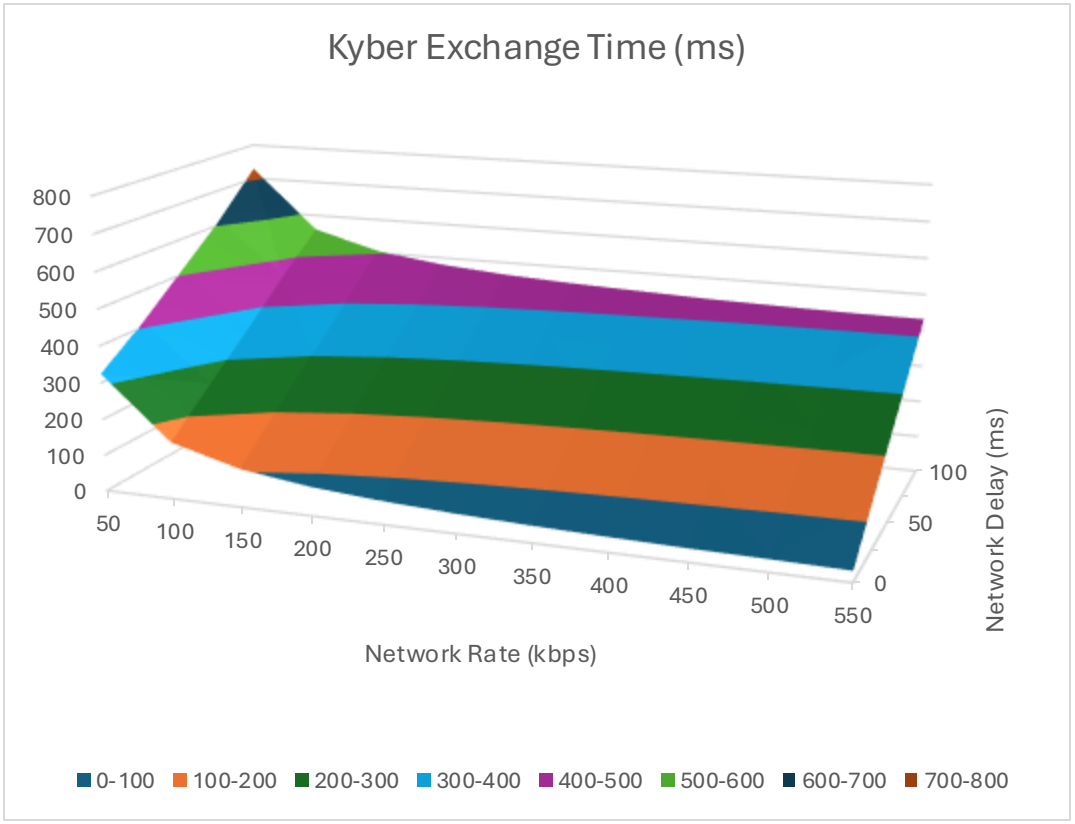
```
joshua@SMAUG:~/ECE4301-Final$ sudo ./run-client.sh target/debug/ECE4301-Final 4kbit 0ms client
Diffie-Hellman exchange time: 992.670 ms
joshua@SMAUG:~/ECE4301-Final$
```

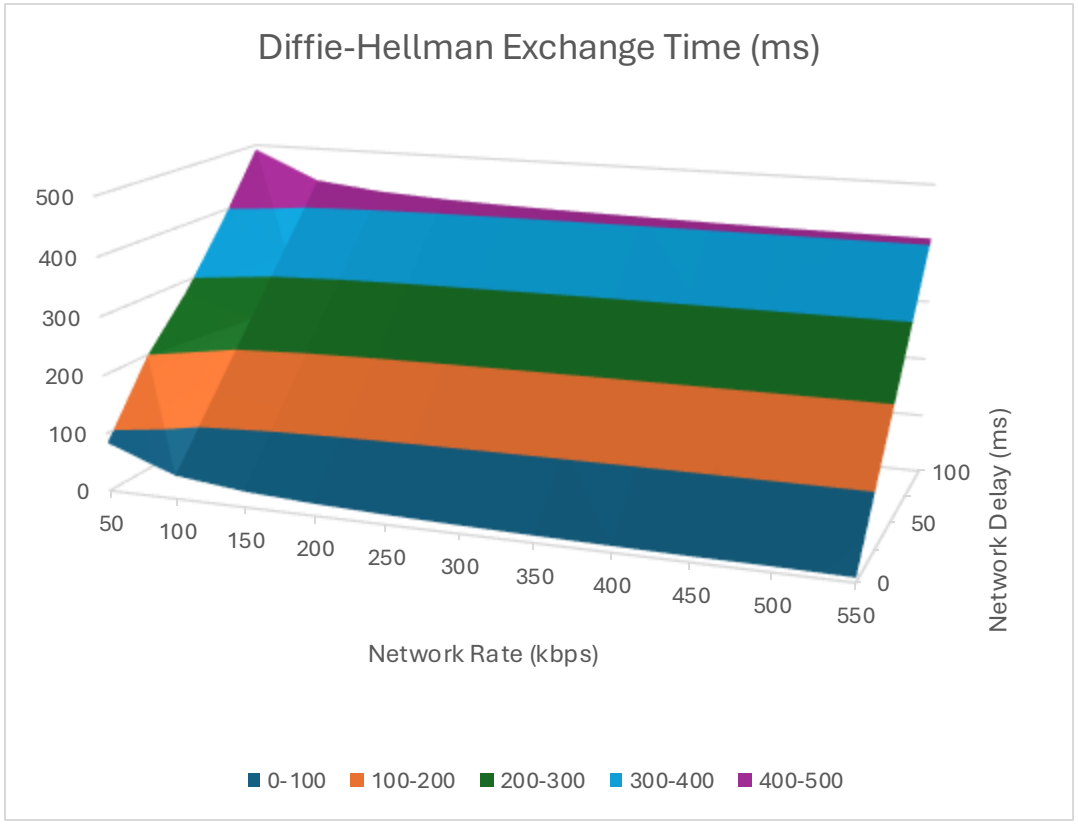
Parses output with Regular Expressions

Network Delay (ms)	Network Rate (kbps)	Kyber Exchange Time (ms)	Diffie-Hellman Exchange Time (ms)	Difference (ms)	Ratio
0	50	324.307	83.551	240.756	3.8815454
0	100	156.39	40.297	116.093	3.88093406
0	150	104.758	27.144	77.614	3.85934276
0	200	78.965	20.562	58.403	3.84033654
0	250	63.49	16.635	46.855	3.81665164
0	300	53.154	14	39.154	3.79671429
0	350	45.776	12.114	33.662	3.77876837
0	400	40.248	10.708	29.54	3.7586851
0	450	35.979	9.61	26.369	3.74391259
0	500	33.448	9.519	23.929	3.51381448
0	550	29.924	8.015	21.909	3.73349969
25	50	412.148	198.92	213.228	2.07192841
25	100	257.326	140.809	116.517	1.82748262
25	150	205.701	127.67	78.031	1.61119292
25	200	179.917	121.097	58.82	1.48572632

Key Exchange Times

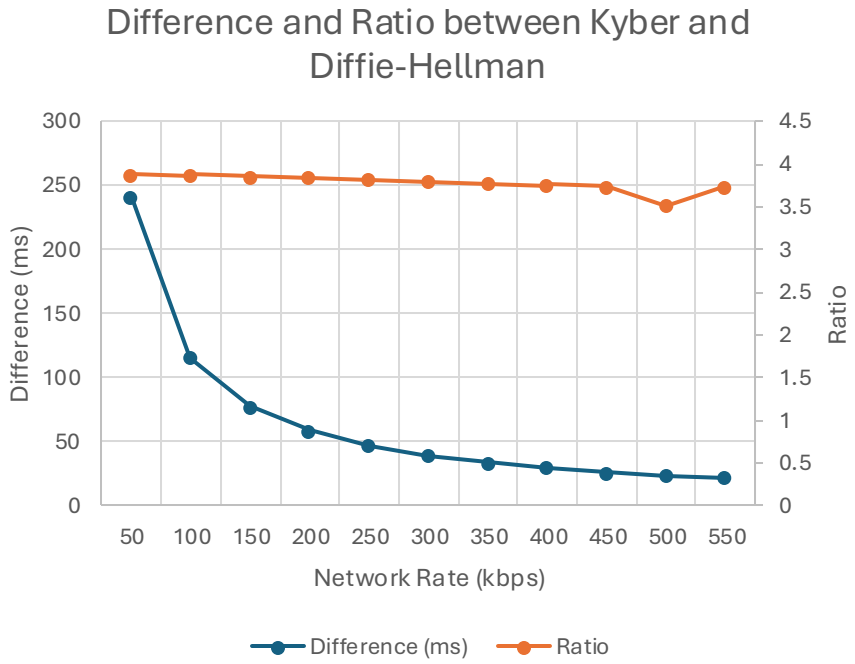
- Inversely proportional to network rate
- Directly proportional to delay (latency)
- Similar response when latency-limited
  - Bandwidth is the interesting variable





Network Rate Analysis

- Ratio between Diffie-Hellman and Kyber key exchange time is mostly independent from network rate
- Approximately 3.8x more data needed for Kyber
- At 550 kbps, the difference is about 22 ms.



Where does it matter?

Domain	Network Characteristics	Evaluation
GSO Satellite	10kbit—220mbit, 500ms delay	No difference at 8mbit (2s), small difference at 10kbit: ~2.4s vs ~3.5s. PQ transition is cheap
Zigbee (IoT)	1kbit—250kbit, 1ms delay	2kbit: 3.7s vs 9.5s; 20kbit: 242ms vs 823ms; 200kbit: 25ms vs 83 ms. PQ transition is expensive
Internet	512kbit—1gbit, 10-150ms delay	3G (512kbit, 150ms): 609ms vs 633 ms; Gigabit ethernet (1gbit, 10ms) 41 ms vs 43 ms. PQ transition is cheap

Questions?